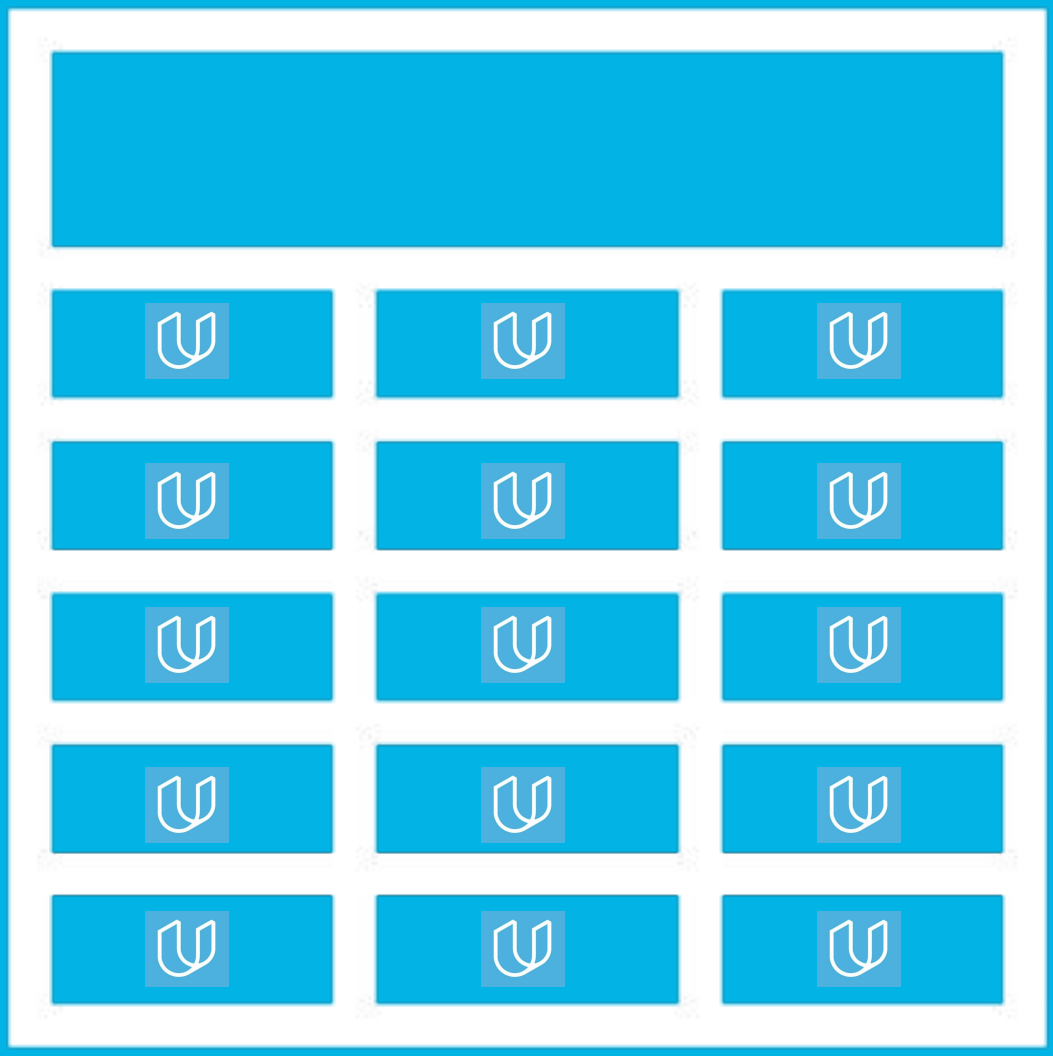
TimeSheets:
Threat Report



YOUR NAME:

Mahmoud Sobhy

27/2/2025

Purpose of this Report:

This is a threat model report for **TimeSheets**. The report will describe the threats facing TimeSheets. The model will cover the following:

- Threat Assessment
    - Scoping out Asset Inventory
    - Architecture Audit
    - Threat Model Diagram
    - Threats to the Organization
    - Identifying Threat Actors
- Vulnerability Analysis
- Risk Analysis
- Mitigation Plan

Section 1

Initial Threat Assessment

Completed Asset Inventory

Components and Functions

- ***TimeSheets Web Server:*** The web server's primary role is to serve static content to a requesting client through the http protocol.
- ***TimeSheets Application Server:*** The application server handles all the business logic process and serves dynamic content.
- ***TimeSheetsDB:*** The database server stores employee data and will be queried from the application server.
- ***AuthDB:*** Stores user authentication data (credentials) and will be queried from the application server.

Completed Asset Inventory

Overview of Application Functionality

TimeSheets is used by employees to track their hours worked. Users will login to the TimeSheets application from their device.

Data Flow

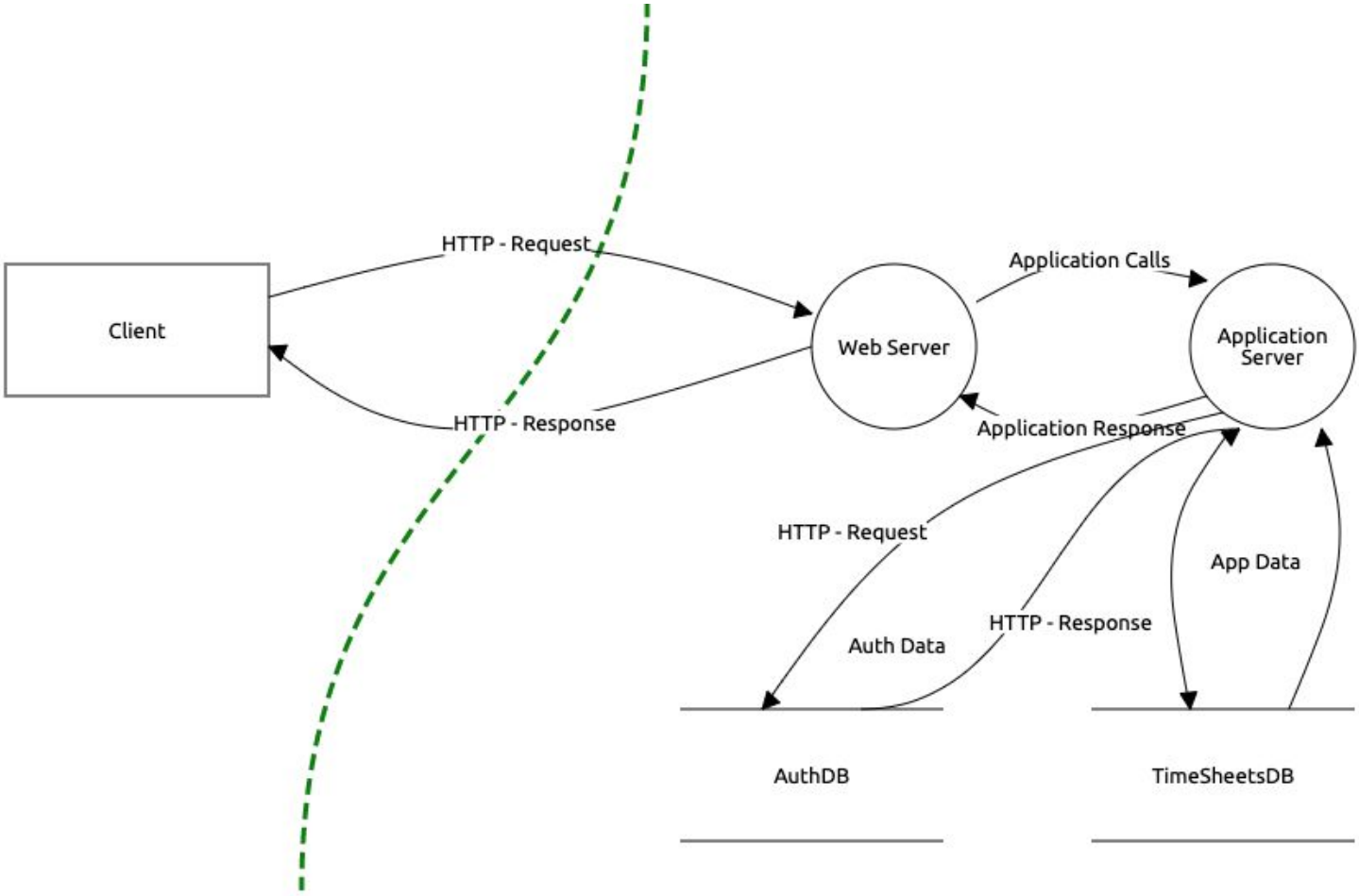
Request is generated from the client via the Internet. The request arrives at the TimeSheets web server which serves static content to the user (HTML, images, etc). Dynamic data is retrieved from the database and served to the client.

Completed Architecture Audit

Flaws

- *There is a lack of encryption at rest - database servers are storing data on unencrypted disks.*
- *There is lack of redundancy.*
- *There is no firewall that is filtering traffic coming from the Internet*

Completed Threat Model



- Employee Data Unencrypted at Rest
- Authentication data is using reversible encryption
- Authentication requests are not encrypted in transit
- Sensitive data is encrypted using DES algorithm

Completed Threat Analysis

What Type of Attack Caused the Login Alerts?

Man in the Middle (MitM)

What Proves Your Theory?

There is lack of encryption between the client and the application. A malicious actor is sniffing traffic and intercepting the requests with a valid username/password in the request. Additionally, the logs show successful login attempts from the expected IP, but also a different location at the same time.

Completed Threat Actor Analysis

Who is the Most Likely Threat Actor?

Internal User

What Proves Your Theory?

The IP address of the unexpected login matches that of an internal user. Additionally, there was no data leaked from the company, and no data changed. Just data accessed that the legitimate user typically doesn't access.

Section 2

Vulnerability Analysis

2.1 Employee Data Unencrypted at Rest

Discovery:

During threat modeling, the Site Reliability Engineering (SRE) team confirmed that the database is on a server that does not have encryption at rest.

Why is this an issue?

If the database were to be leaked for any reason, all of the data would be in **plaintext.** This can be executed by a hacker, who will have access to all employee data.

2.2 Authentication Data Stored Using Reversible Encryption

Discovery:

During threat modeling, the Database Administrators (DBA) team confirmed that the database is storing authentication data (credentials) encrypted.

Why is this an issue?

Authentication data should be **hashed,** not **encrypted.** Reversible Encryption is, well, reversible. If an attacker manages to get the private key, all authentication data will turn into plaintext. Hashing (with salting) ensures that authentication data will be useless if leaked.

2.3 Authentication Requests are Unencrypted in Transit

Discovery:

During threat modeling, the security team confirmed that authentication requests are being transmitted in plaintext.

Why is this an issue?

This exposes authentication requests to **MiTM (Man-in-The-Middle)** attacks, since no encryption in transit is used, an attacker can easily sniff the traffic, modify it, or just look at it.

2.4 DES Algorithm in Use

Discovery:

While conducting threat modeling, the security team identified sensitive data being stored using the Data Encryption Standard (DES) algorithm.

Why is this an issue?

DES is deprecated. It is vulnerable to **bruteforce attacks** due to its 56-bit key size, which allows an attacker with patience and appropriate hardware to crack the key.

Optional Task:

Examine the threat model diagram from Section 1 and answer:

What non-encryption issues can you identify?

What recommendation would you give to solve those issues?

Why do you recommend those solutions?

- *Absence of Firewalls and IPS, which allows hackers to make malicious requests or access the network's infrastructure easily, an NGFW (Next-Gen Firewall) includes a Firewall and an IPS which will solve this.*
- *Absence of backup servers, which is catastrophic in case of a ransomware attack or accidental database deletion.*
- *Absence of a central monitoring and logging solution, an SIEM will suffice for this,*

Section 3

Risk Analysis

3.1 Scoring Risks

Risk	**Score** ***(1 is most dangerous, 4 is least dangerous)***
Unencrypted at Rest	2
Reversible Encryption	4
Unencrypted in Transit	1
Outdated Algorithm	3

3.2 Risk Rationale

Why did you choose that ranking? Make sure to include your risk ranking methodology. Your explanations must be based on the learnings from this course as well as observations from the initial report. *(Did you use a tool or defined risk scoring system?)*

Using CVSS (Common Vulnerability Scoring System),

1) **Unencrypted in Transit (Likelihood: Critical - Impact: Critical):**

   This is very risky, since anyone who logs in remotely from a public place will be vulnerable to **Man-in-The-Middle attacks.**

2) **Unencrypted at Rest (Likelihood: High - Impact: Critical):**

   This vulnerability is less dangerous since it requires the attacker to have **read** access to the database.

3) **Outdated Algorithm (Likelihood: Medium - Impact: Critical):**

   DES is susceptible to bruteforce attacks, but they take time and require the attacker to have **read** access to the database. It is still risky, but better than no encryption at all.

4) **Reversible Encryption (Likelihood: Low - Impact: Critical):**

   Exploiting this requires the attacker to have the **private key,** and **read** access to the database. It is not that risky.

Section 4

Mitigation Plan

4.1 Employee Data unencrypted at Rest

What is Your Recommended Mitigation Plan?

Using symmetric next-generation encryption algorithms (according to CISCO's recommendations) such as AES-GCM-256.

Why Did you Recommend This Course of Action?

This will encrypt the data with a private key, so if an attacker manages to retrieve the database, it will be useless without the private key. AES-GCM-256 is very secure, and quantum resistant for the future.

4.2 Authentication Data Stored using Reversible Encryption

What is Your Recommended Mitigation Plan?

Hashing the authentication data using SHA-512 with salting.

Why Did you Recommend This Course of Action?

This ensures that authentication data will be useless if a hacker manages to gain access to it. The use of Salt will prevent rainbow table attacks as well. SHA-512 is very secure and will likely remain secure for very long.

4.3 Authentication Requests are Not Encrypted in Transit

What is Your Recommended Mitigation Plan?

Using TLS 1.3 to encrypt the data in transit.

Why Did you Recommend This Course of Action?

TLS 1.3 will ensure that the data in transit will be encrypted, making Man-in-The-Middle attacks practically useless. TLS uses Asymmetric encryption to transmit the symmetric secret key, then the client and the server use the symmetric secret key, ensuring speed and security.

4.4 DES Algorithm in Use

What is Your Recommended Mitigation Plan?

Using AES-GCM-256 instead of DES.

Why Did you Recommend This Course of Action?

DES is insecure. It is vulnerable to bruteforce attacks especially in single-layer encryption, and vulnerable to Meet-in-The-Middle attacks in multi-layer encryption (shortens the time needed for bruteforcing). AES-GCM-256 is much more secure, and likely not vulnerable to bruteforce attacks in the future.

4.5 Security Audit

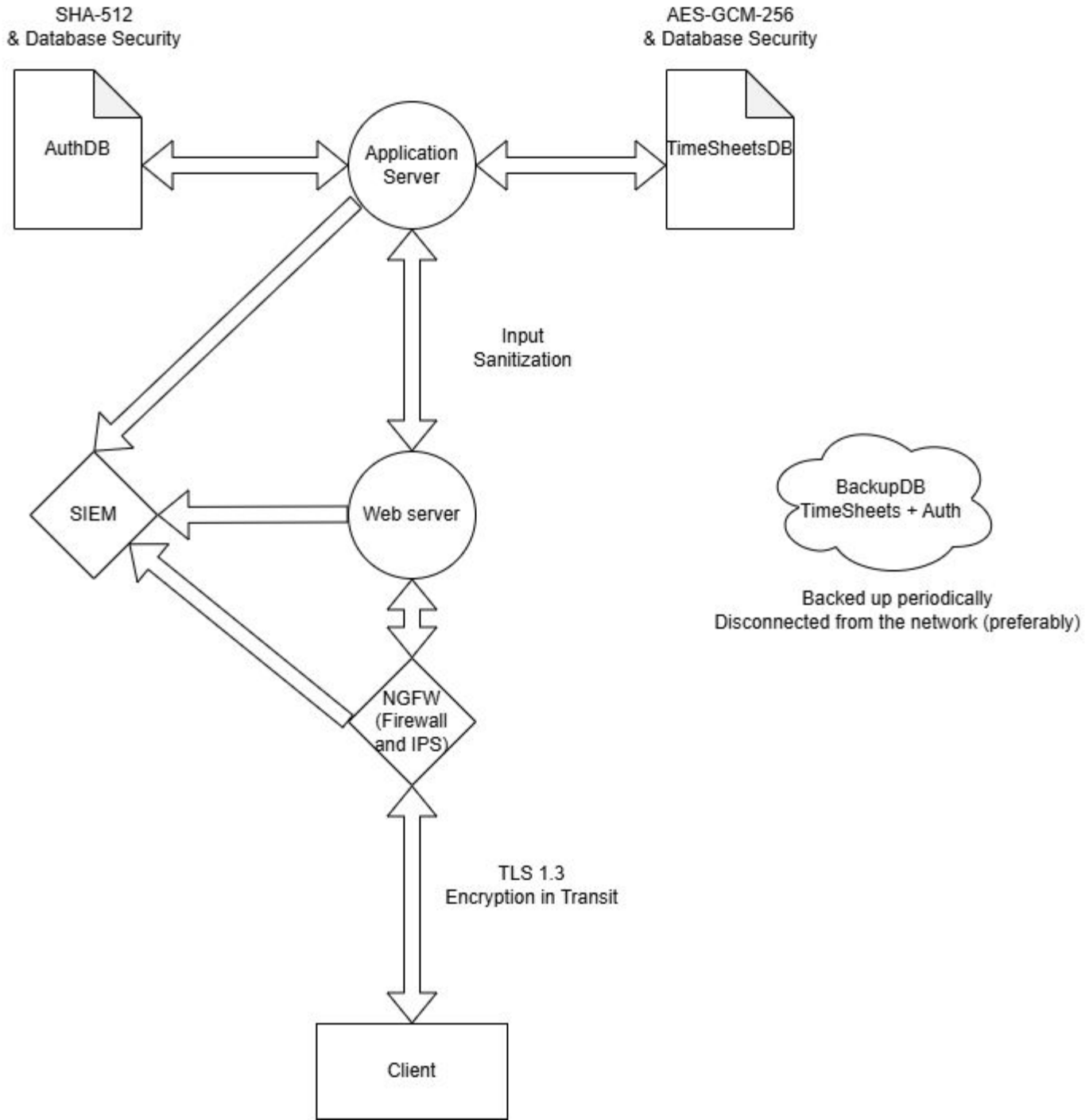
The audit team has been made aware of the systemic issue and wants to ensure your recommendations are followed. What steps can the audit team take?

- **Password Security Verification** – Implement a structured approach to test and confirm that all stored passwords are securely hashed using strong cryptographic algorithms, such as SHA-256 or bcrypt. Regularly audit password storage practices to ensure compliance with security standards.
- **Secure Communication Enforcement** – Verify that HTTPS is consistently enforced across all communication channels to prevent data interception. Conduct periodic security assessments to confirm that SSL/TLS certificates are valid, up to date, and properly configured.
- **Encryption Algorithm Upgrade** – Ensure that outdated encryption algorithms such as DES have been fully replaced with stronger alternatives like AES-256. Conduct system-wide reviews to identify any legacy encryption implementations and update them accordingly.
- **Regular Security Audits** – Establish periodic encryption audits and penetration testing to proactively identify vulnerabilities in encryption practices. Implement a continuous monitoring system to detect any deviations from security policies.
- **Compliance and Best Practices** – Align encryption practices with industry standards such as NIST, OWASP, and ISO 27001. Maintain thorough documentation of encryption policies and provide training to development teams to reinforce security best practices.

Optional Task:

Create an architecture diagram of a secure system.

Image of your secure architecture:



Optional Task *(Continued)*:

Additional Steps Would You Recommend to Prevent the Attack as well as Future Issues:

- Hiring of Cybersecurity professionals in teams (red team, blue team, GRC and incident response)
- Following NIST standards for server security (SP 800-123)
- Hosting a bug bounty program (Optional; to help find vulnerabilities)
- Frequent **external** audits from trusted organizations, and Pentesting as a Service (PTaaS)
- Writing guidelines for data storage (PII, authentication data, etc.)
- Redundancy in case of attacks, through frequent backups.
- Updating systems automatically through tools like **WSUS**